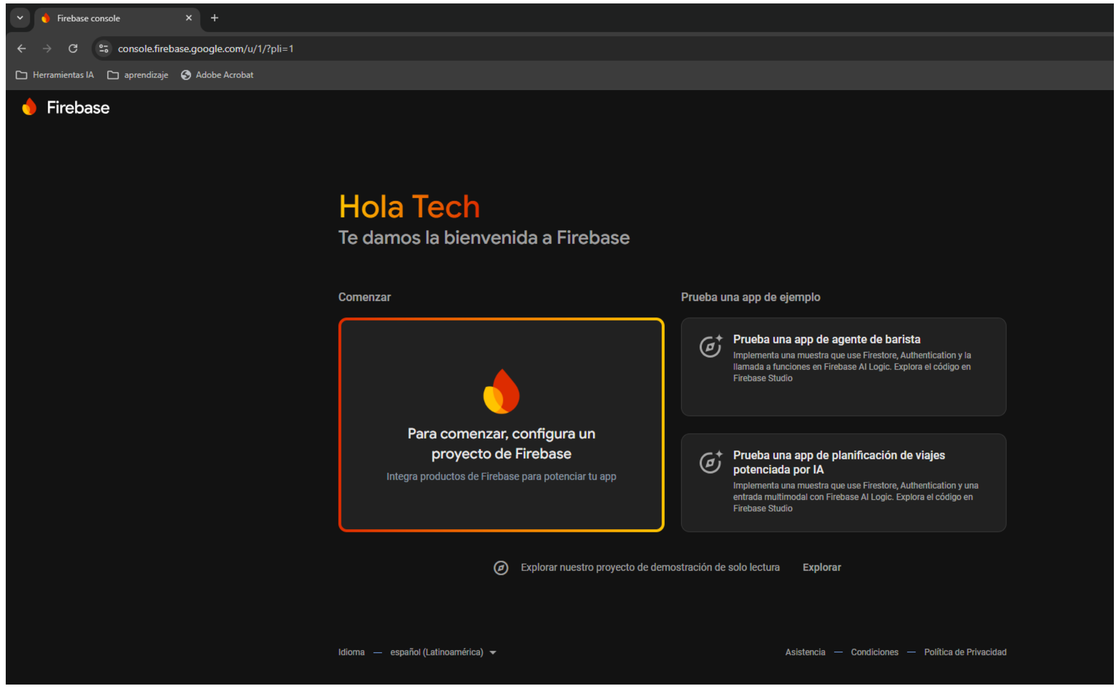
Web y Patrones
Semana 4

Firebase y Crud

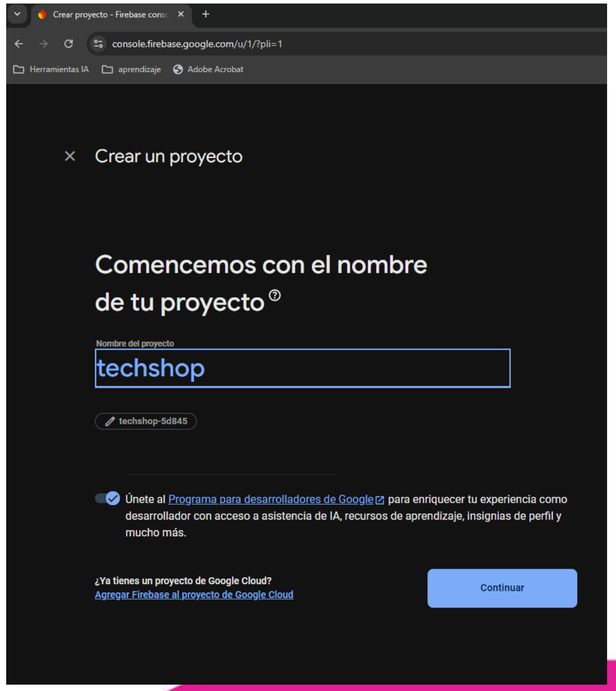
- Creamos nuestra cuenta en Firebase para poder utilizar el espacio “storage” en el almacenamiento de imágenes de nuestro sitio web
- Se ingresa a https://console.firebase.google.com/ mediante algún usuario de Google.

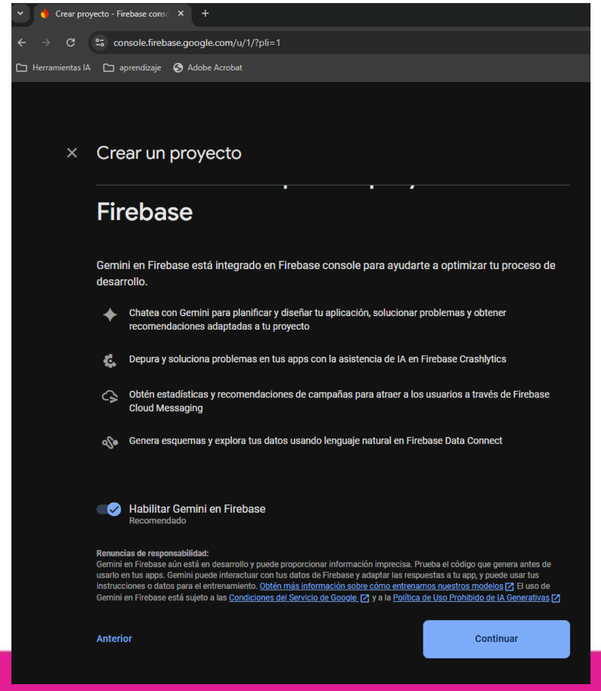


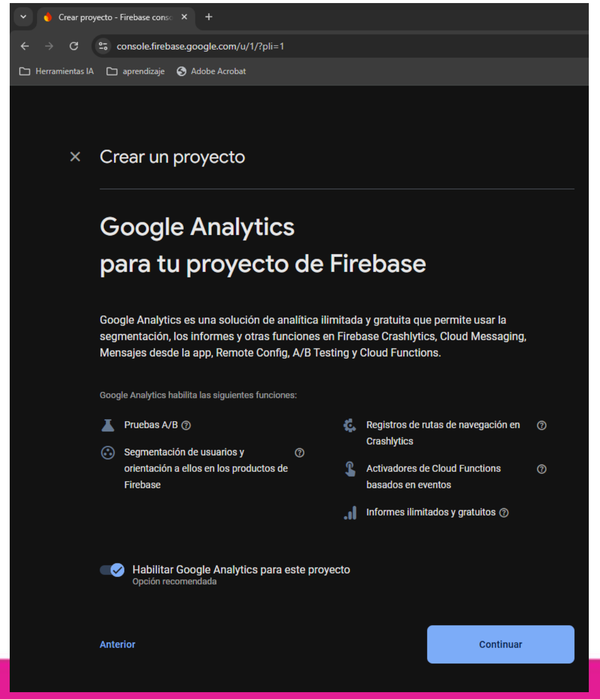
fidélitas
Universidad

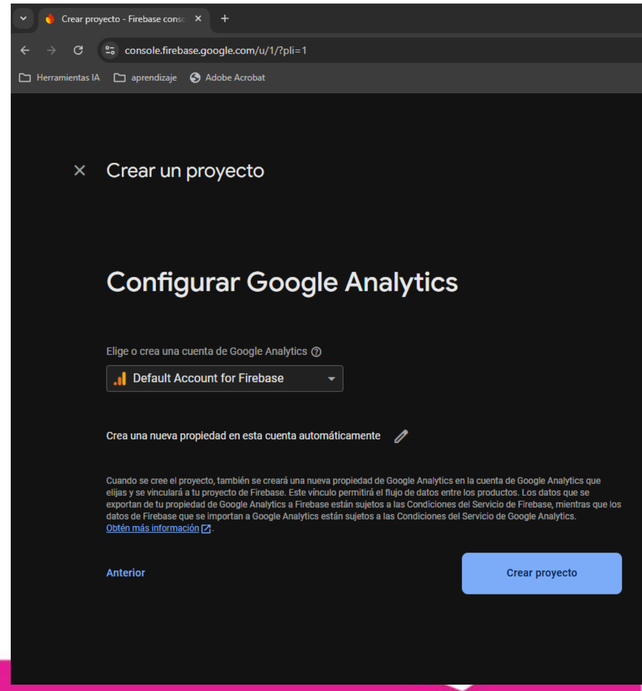
Firebase y CRUD

- Se crea el proyecto techshop.



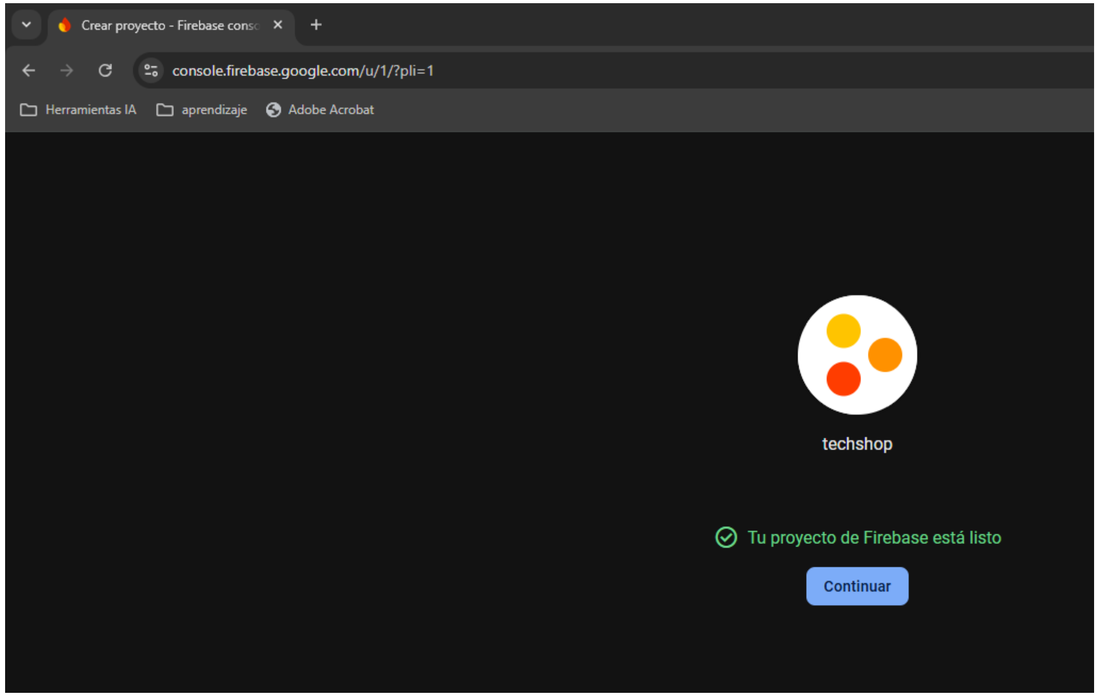






fidélitas Universidad

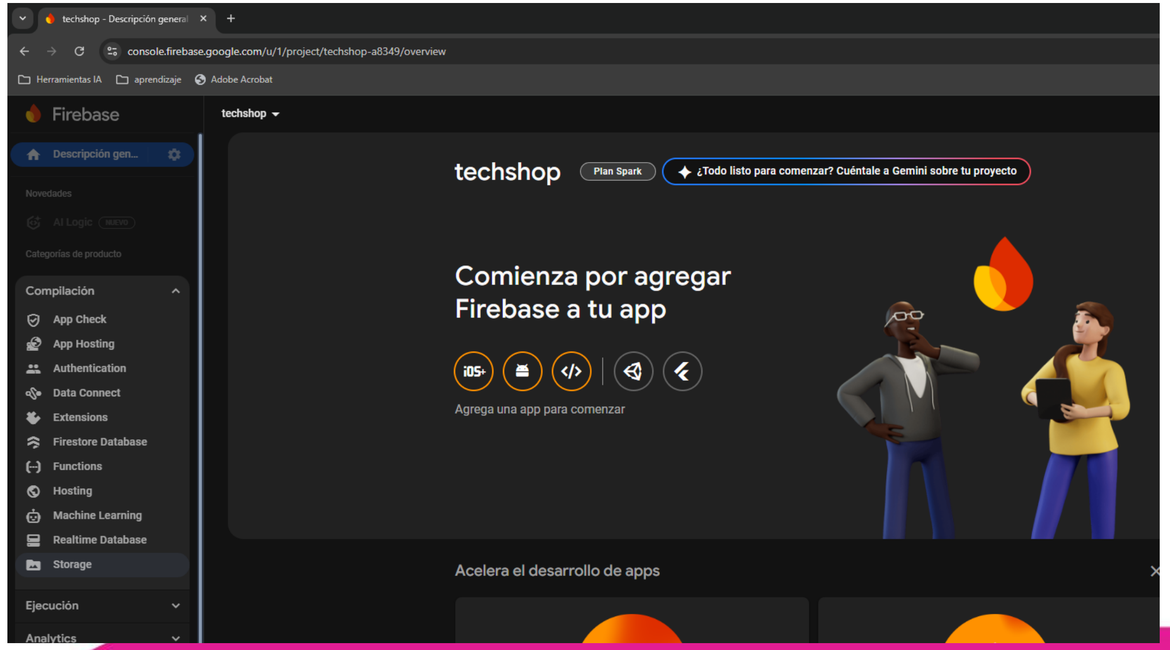
Firebase y CRUD

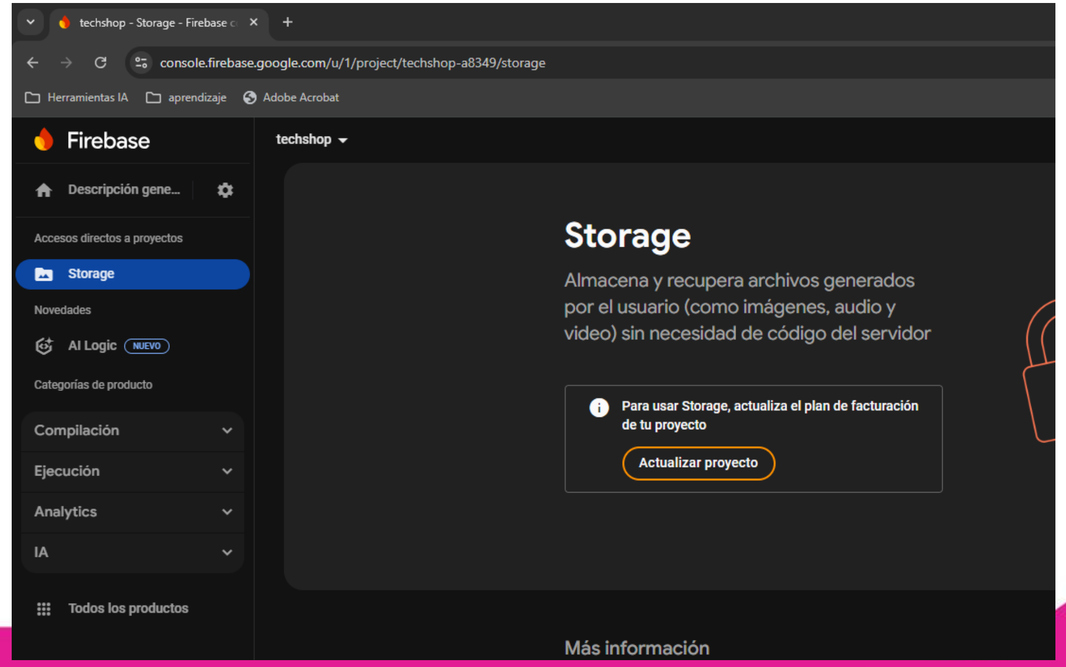


Se habilita el Storage

En la sección de “Compilación” se da click en Storage.

Hay que activar el plan de facturación (no se genera un cobro)

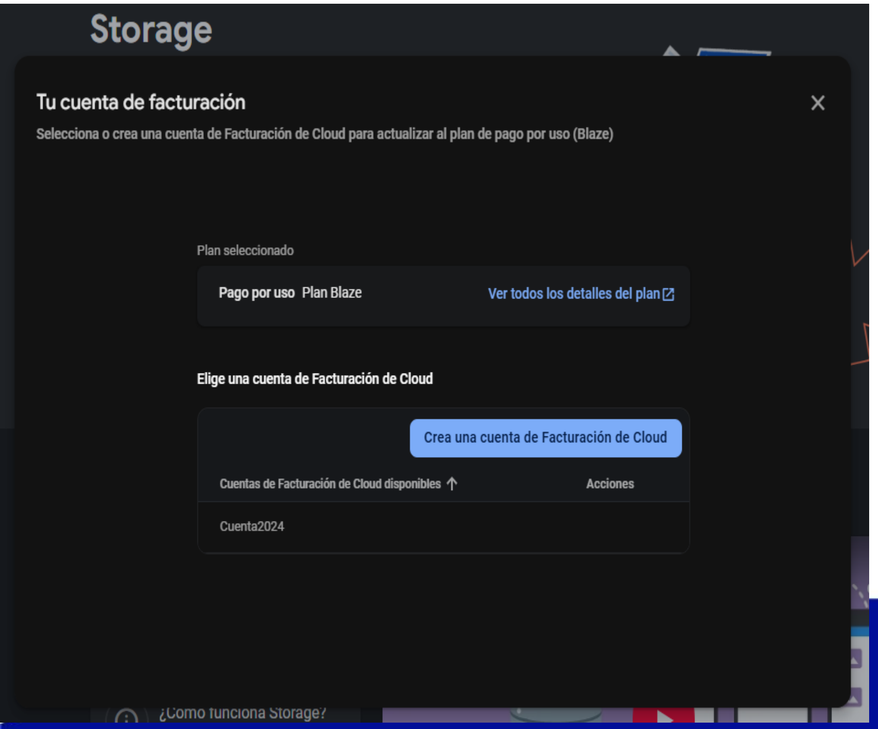


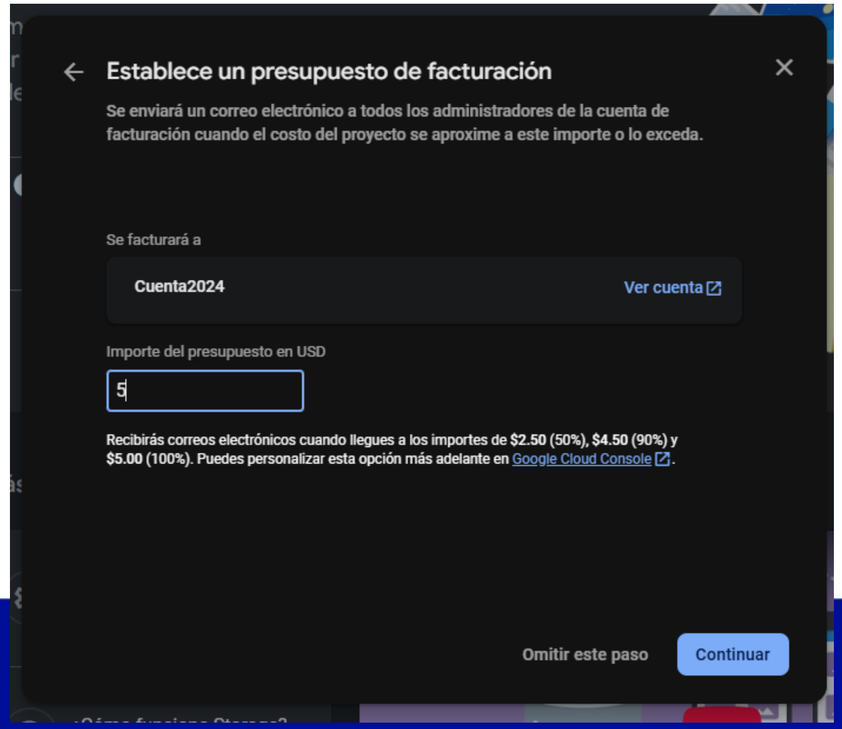


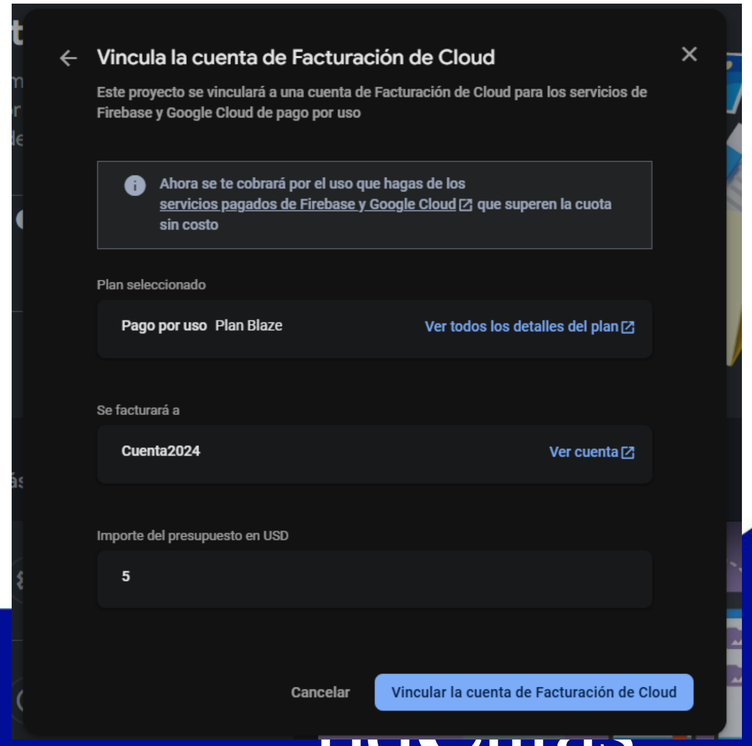
fidélitas Universidad

Cuenta de Facturación

Como Firebase se utiliza a nivel comercial, la creación de la cuenta de facturación es un medio que utiliza Google para recuperar posible clientes comerciales. Lo que se realizará en el curso no genera ningún cargo.

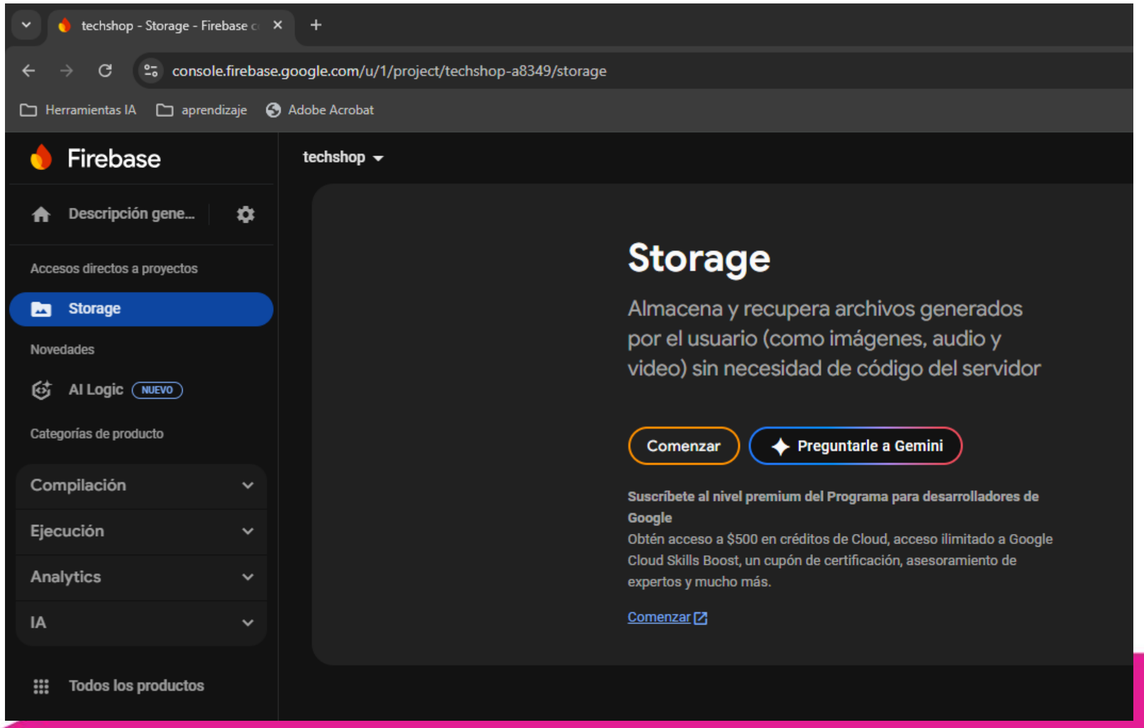


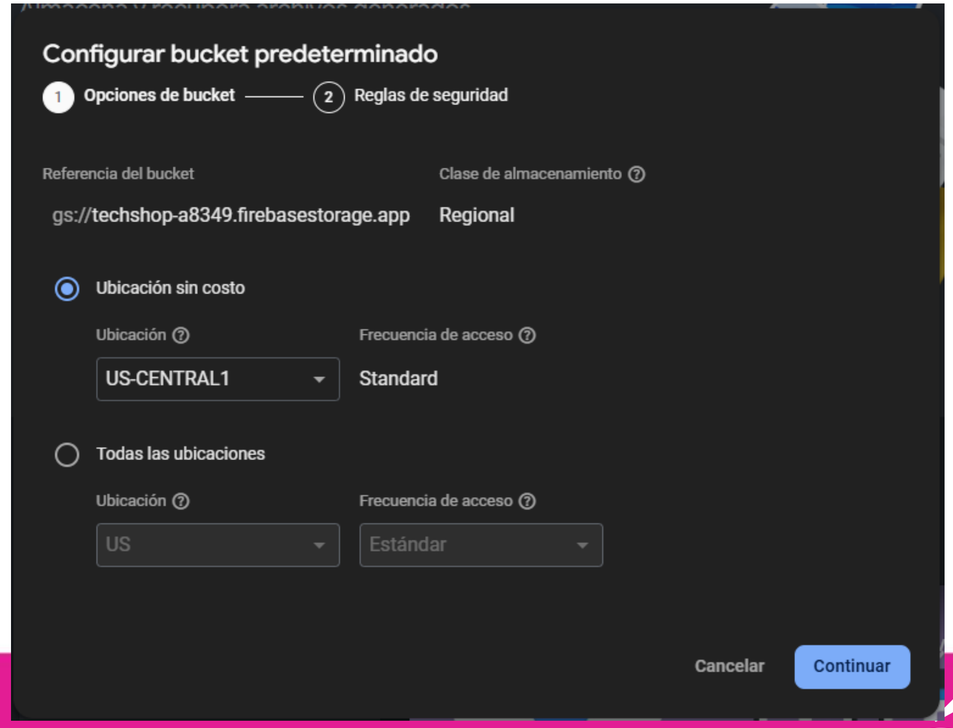




Comenzar y configurar bucket sin costo

Se da la opción de comenzar el Storage y si utiliza una ubicación sin costo.

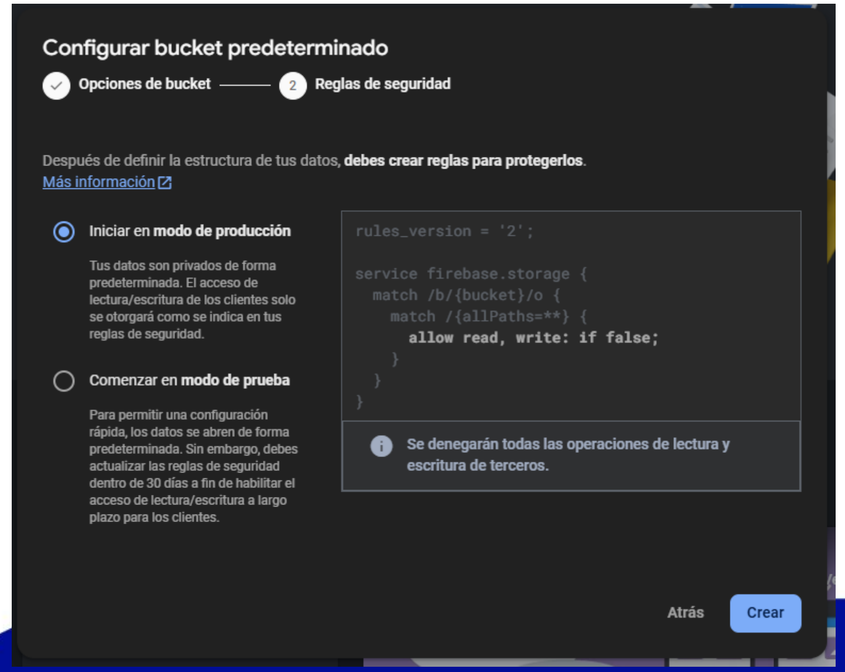


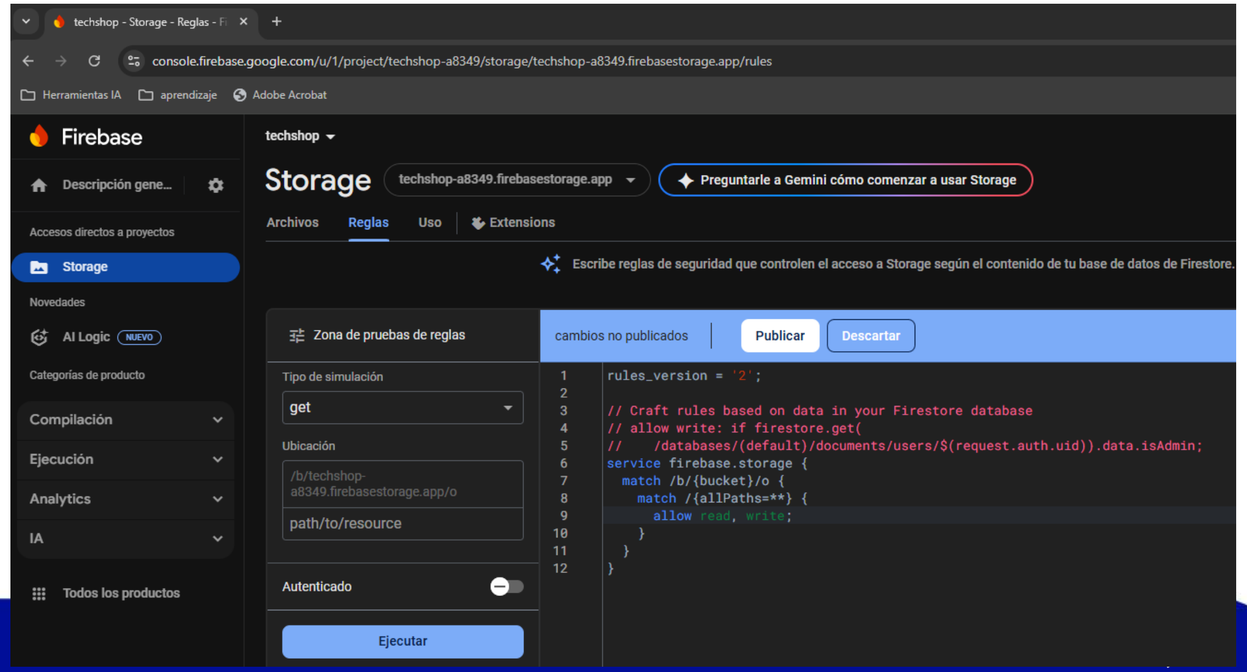


fidélitas Universidad

Definición de reglas públicas

Se deja la opción de iniciar modo de producción y luego se hacen públicas las reglas. Observen que se modifica la línea 9 de las reglas. Originalmente estaba “allow read, write: if false” y queda “allow read, write”.

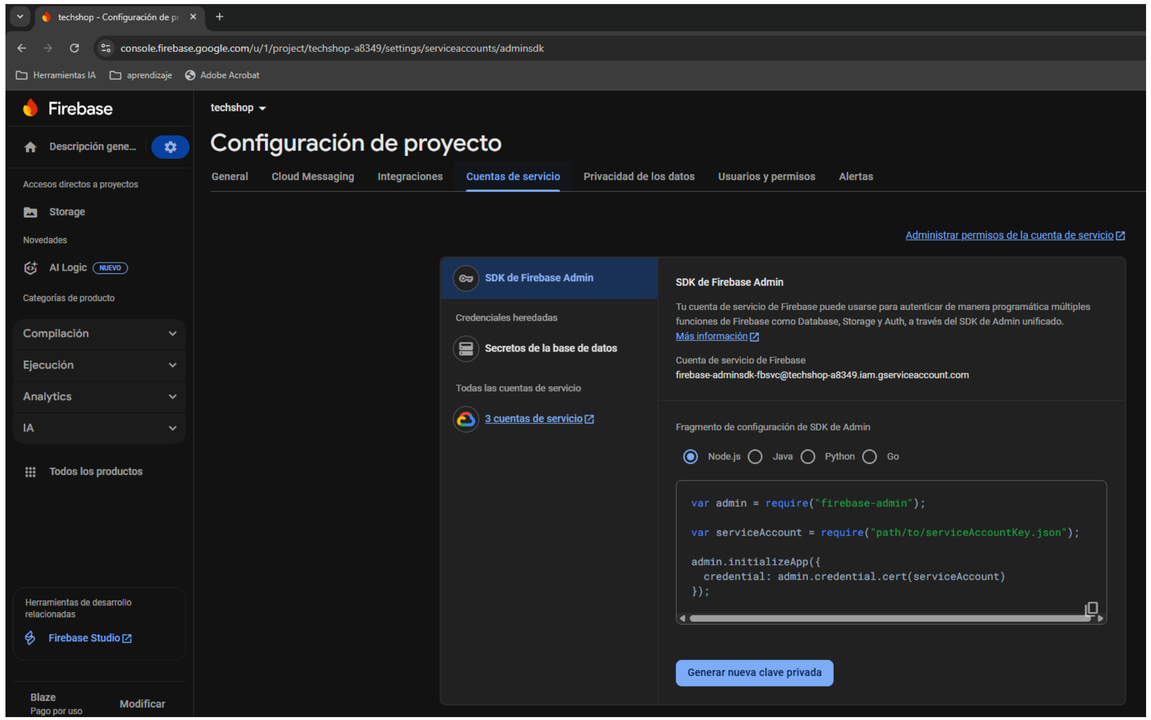




fidelitas Universidad

Generación de clave privada

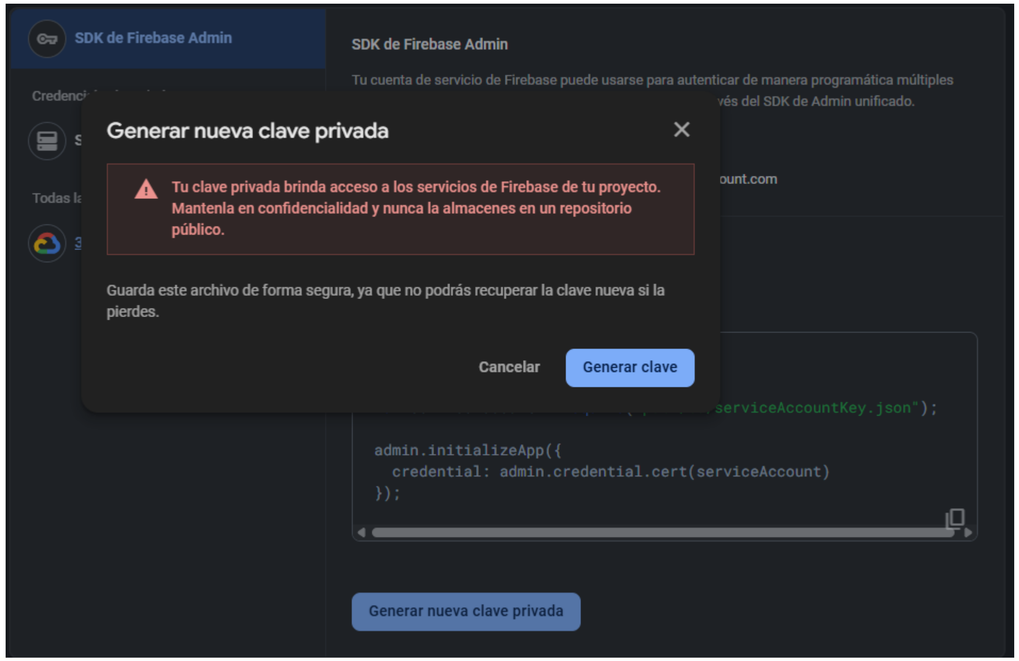
Se genera una clave privada para colocar en el proyecto tienda

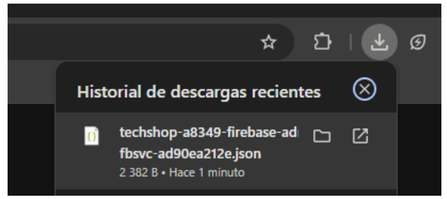


fidélitas Universidad

Generación y descarga

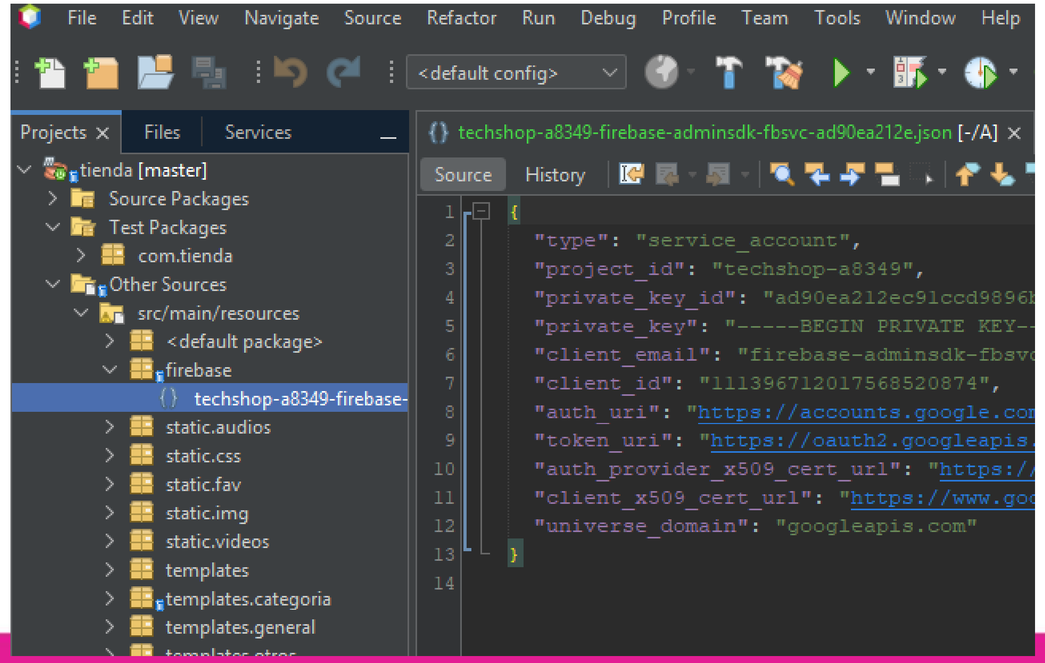
Se genera la clave privada y se debe descargar dentro del proyecto tienda, debe ser un lugar seguro





Folder firebase

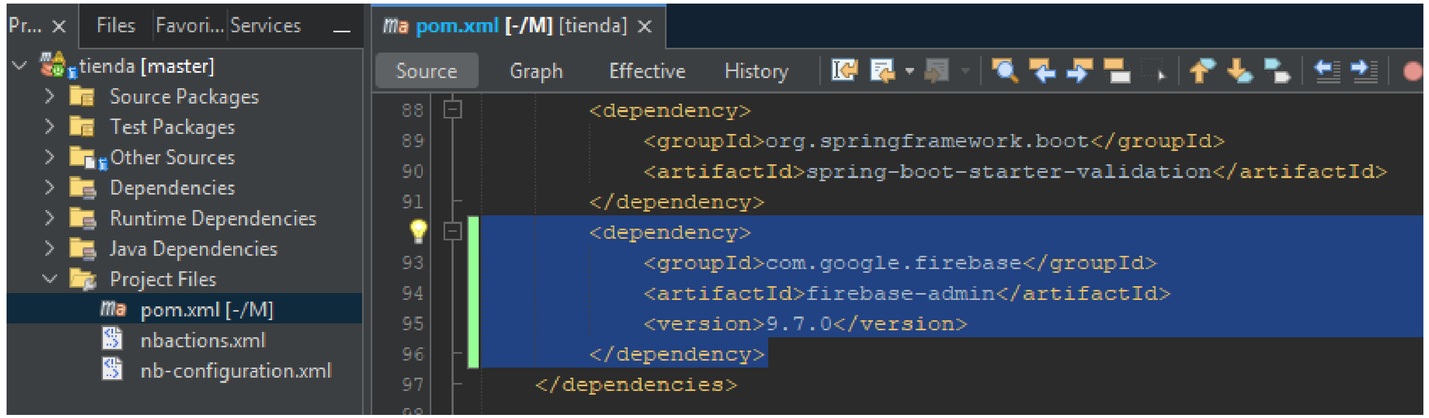
Se crea un nuevo folder en la raíz de “default package”, llamado “Firebase”, ahí, colocamos el archivo descargado.



fidélitas Universidad

pom.xml

Se incluye la dependencia de Firebase en el archivo pom.xml, recuerde hacer un “Clean and Build” (martillo y escoba)

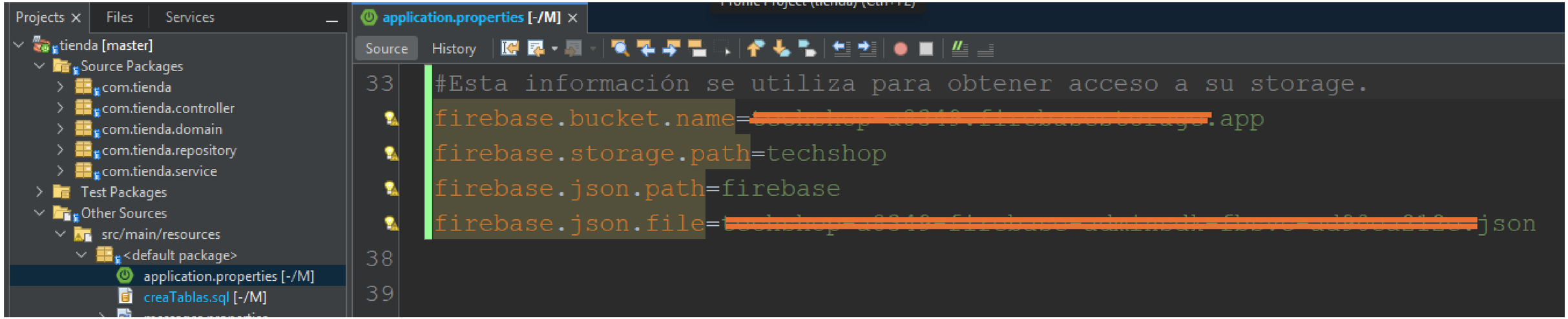


Recuerda: “Martillo y Escoba”

fidélitas Universidad

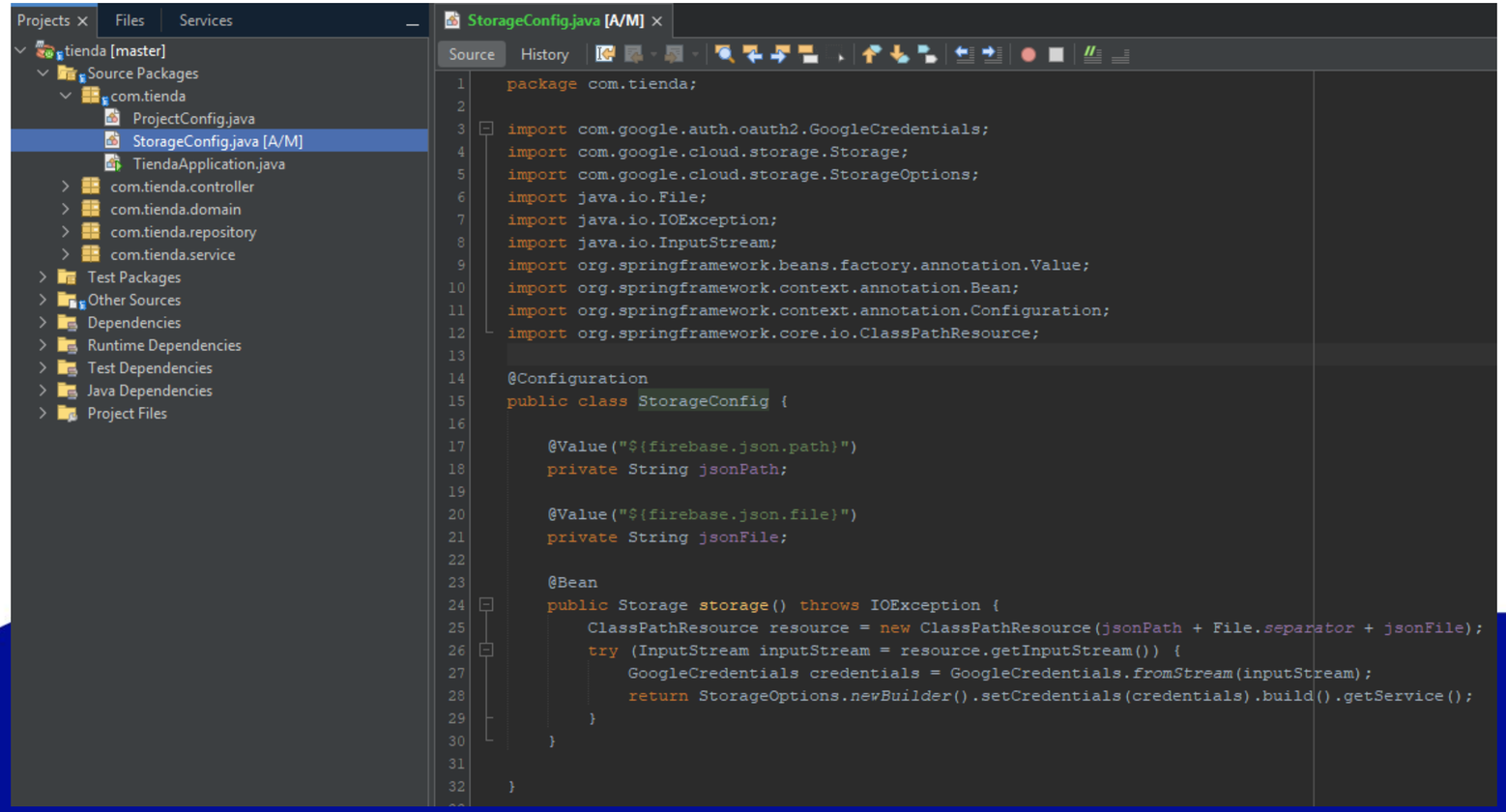
FirebaseStorageService

Agreguemos a application.properties 4 valores que se utilizarán en el la comunicación con firebase



FirebaseStorageService

Se crea una nueva clase StorageConfig para gestionar las credenciales del Storage

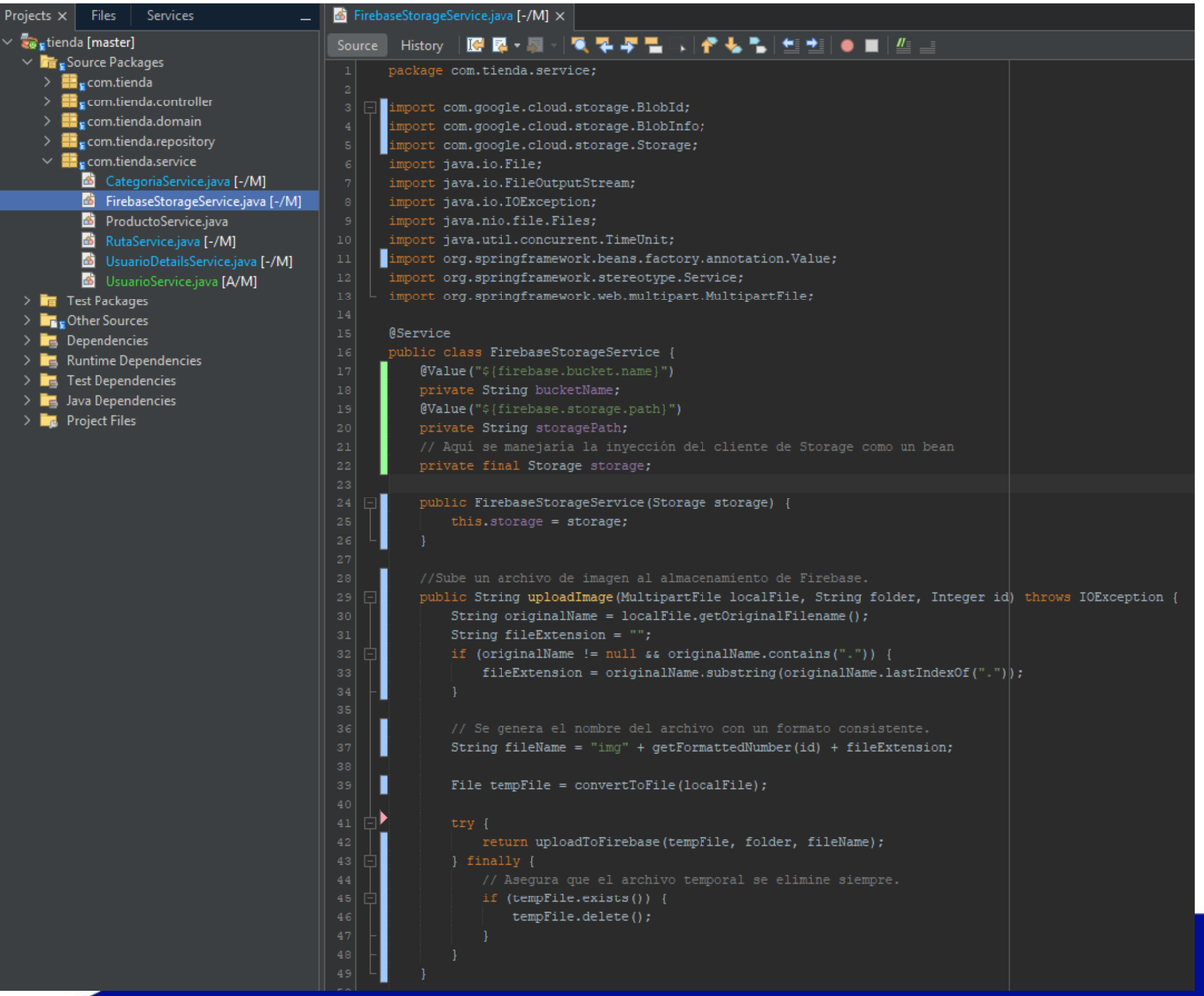



```java
package com.tienda;

import com.google.auth.oauth2.GoogleCredentials;
import com.google.cloud.storage.Storage;
import com.google.cloud.storage.StorageOptions;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;

@Configuration
public class StorageConfig {

    @Value("${firebase.json.path}")
    private String jsonPath;

    @Value("${firebase.json.file}")
    private String jsonFile;

    @Bean
    public Storage storage() throws IOException {
        ClassPathResource resource = new ClassPathResource(jsonPath + File.separator + jsonFile);
        try (InputStream inputStream = resource.getInputStream()) {
            GoogleCredentials credentials = GoogleCredentials.fromStream(inputStream);
            return StorageOptions.newBuilder().setCredentials(credentials).build().getService();
        }
    }

}
```

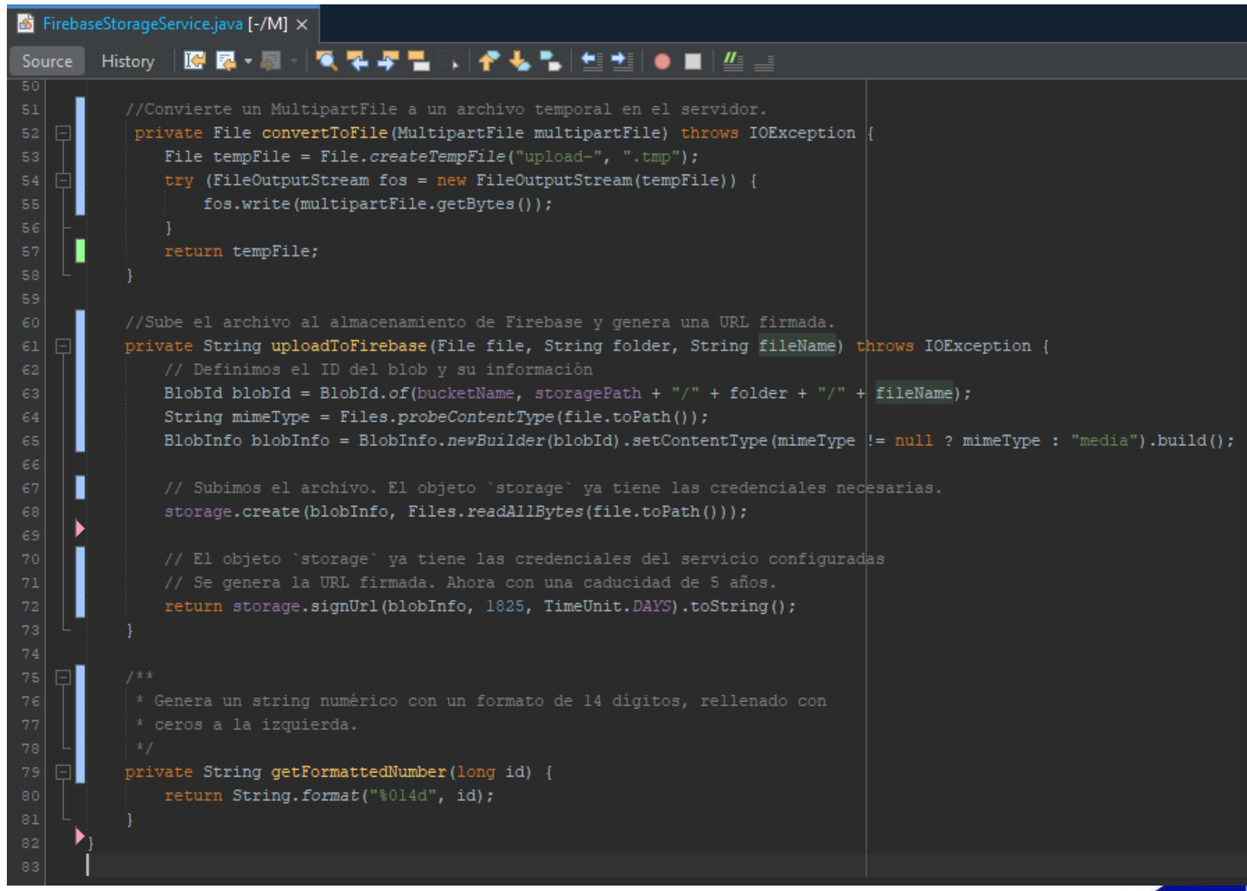
fidélitas Universidad

FirebaseStorageService

Descargue de los recurso la clase FirebaseStorageService.java, los ajustes se realizan a application.properties




```java
package com.tienda.service;

import com.google.cloud.storage.BlobId;
import com.google.cloud.storage.BlobInfo;
import com.google.cloud.storage.Storage;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.nio.file.Files;
import java.util.concurrent.TimeUnit;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

@Service
public class FirebaseStorageService {
    @Value("${firebase.bucket.name}")
    private String bucketName;
    @Value("${firebase.storage.path}")
    private String storagePath;
    // Aquí se manejaría la inyección del cliente de Storage como un bean
    private final Storage storage;

    public FirebaseStorageService(Storage storage) {
        this.storage = storage;
    }

    //Sube un archivo de imagen al almacenamiento de Firebase.
    public String uploadImage(MultipartFile localFile, String folder, Integer id) throws IOException {
        String originalName = localFile.getOriginalFilename();
        String fileExtension = "";
        if (originalName != null && originalName.contains(".")) {
            fileExtension = originalName.substring(originalName.lastIndexOf("."));
        }

        // Se genera el nombre del archivo con un formato consistente.
        String fileName = "img" + getFormattedNumber(id) + fileExtension;

        File tempFile = convertToFile(localFile);

        try {
            return uploadToFirebase(tempFile, folder, fileName);
        } finally {
            // Asegura que el archivo temporal se elimine siempre.
            if (tempFile.exists()) {
                tempFile.delete();
            }
        }
    }
```




```java
    //Convierte un MultipartFile a un archivo temporal en el servidor.
    private File convertToFile(MultipartFile multipartFile) throws IOException {
        File tempFile = File.createTempFile("upload-", ".tmp");
        try (FileOutputStream fos = new FileOutputStream(tempFile)) {
            fos.write(multipartFile.getBytes());
        }
        return tempFile;
    }

    //Sube el archivo al almacenamiento de Firebase y genera una URL firmada.
    private String uploadToFirebase(File file, String folder, String fileName) throws IOException {
        // Definimos el ID del blob y su información
        BlobId blobId = BlobId.of(bucketName, storagePath + "/" + folder + "/" + fileName);
        String mimeType = Files.probeContentType(file.toPath());
        BlobInfo blobInfo = BlobInfo.newBuilder(blobId).setContentType(mimeType != null ? mimeType : "media").build();

        // Subimos el archivo. El objeto `storage` ya tiene las credenciales necesarias.
        storage.create(blobInfo, Files.readAllBytes(file.toPath()));

        // El objeto `storage` ya tiene las credenciales del servicio configuradas
        // Se genera la URL firmada. Ahora con una caducidad de 5 años.
        return storage.signUrl(blobInfo, 1825, TimeUnit.DAYS).toString();
    }

    /**
     * Genera un string numérico con un formato de 14 dígitos, rellenado con
     * ceros a la izquierda.
     */
    private String getFormattedNumber(long id) {
        return String.format("%014d", id);
    }
}
```

fidélitas Universidad

CategoriaService.java

Se agregan a la clase, los métodos: get, save y delete

CategoriaService.java [-/M]

Source History

```java
    @Transactional(readOnly = true)
    public Optional<Categoria> getCategoria(Integer idCategoria) {
        return categoriaRepository.findById(idCategoria);
    }

    @Transactional
    public void save(Categoria categoria, MultipartFile imagenFile) {
        categoria = categoriaRepository.save(categoria);
        if (!imagenFile.isEmpty()) { //Si no está vacío... pasaron una imagen...
            try {
                String rutaImagen = firebaseStorageService.uploadImage(
                        imagenFile, "categoria",
                        categoria.getIdCategoria());
                categoria.setRutaImagen(rutaImagen);
                categoriaRepository.save(categoria);
            } catch (IOException e) {

            }
        }
    }

    @Transactional
    public void delete(Integer idCategoria) {
        // Verifica si la categoría existe antes de intentar eliminarlo
        if (!categoriaRepository.existsById(idCategoria)) {
            // Lanza una excepción para indicar que el usuario no fue encontrado
            throw new IllegalArgumentException("La categoría con ID " + idCategoria + " no existe.");
        }
        try {
            categoriaRepository.deleteById(idCategoria);
        } catch (DataIntegrityViolationException e) {
            // Lanza una nueva excepción para encapsular el problema de integridad de datos
            throw new IllegalStateException("No se puede eliminar la categoría. Tiene datos asociados.", e);
        }
    }
}
```

fidélitas Universidad

CategoriaController

Se actualiza el controller para que pueda guardar, eliminar y modificar.

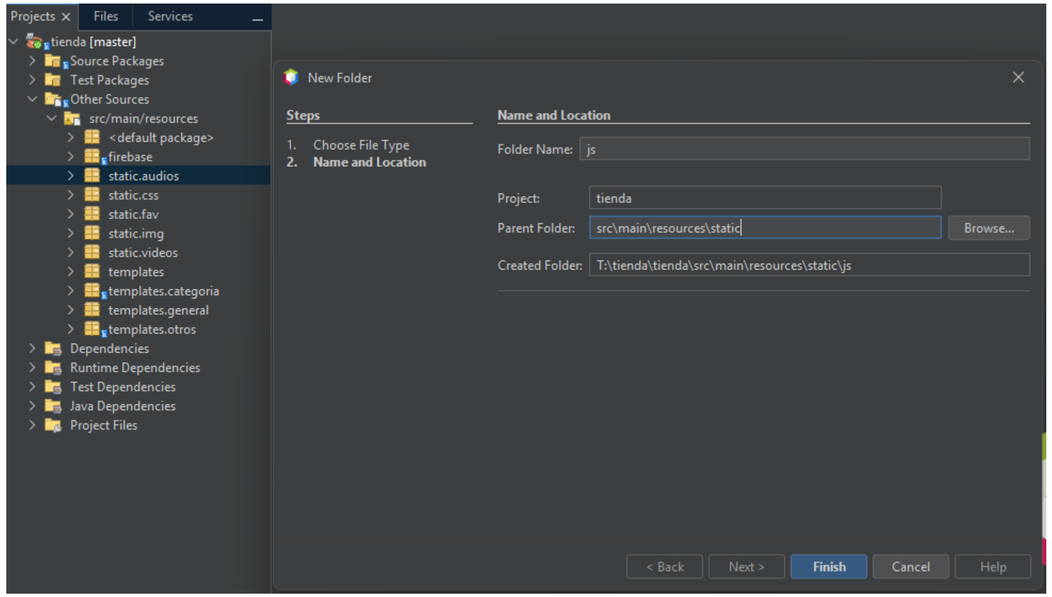
CategoriaController.java [-/M]

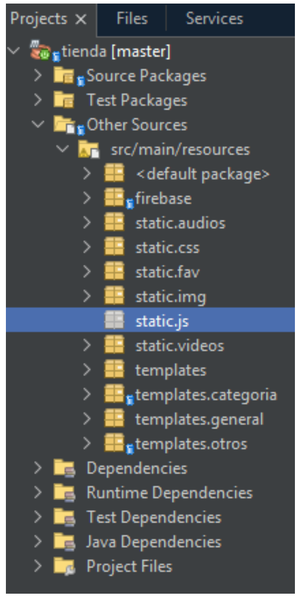
Source History

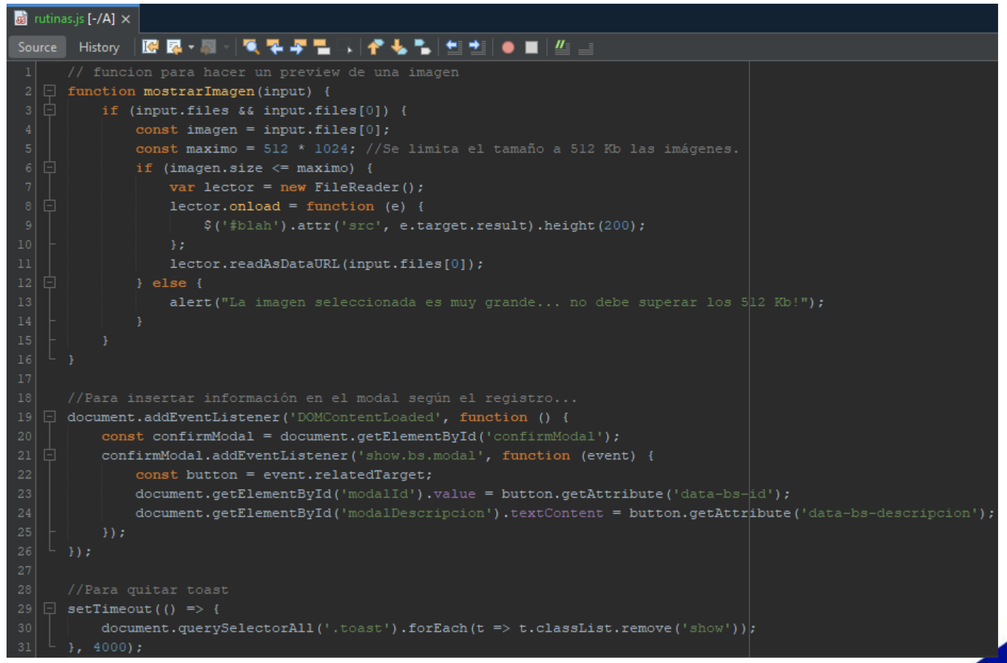
```java
    private final CategoriaService categoriaService;
    private final MessageSource messageSource;

    public CategoriaController(CategoriaService categoriaService, MessageSource messageSource) {
        this.categoriaService = categoriaService;
        this.messageSource = messageSource;
    }

    @GetMapping("/listado")
    public String listado(Model model) {...6 lines }

    @PostMapping("/guardar")
    public String guardar(@Valid Categoria categoria, @RequestParam MultipartFile imagenFile, RedirectAttributes redirectAttributes) {

        categoriaService.save(categoria, imagenFile);
        redirectAttributes.addFlashAttribute("todoOk", messageSource.getMessage("mensaje.actualizado", null, Locale.getDefault()));

        return "redirect:/categoria/listado";
    }

    @PostMapping("/eliminar")
    public String eliminar(@RequestParam Integer idCategoria, RedirectAttributes redirectAttributes) {
        String titulo = "todoOk";
        String detalle = "mensaje.eliminado";
        try {
            categoriaService.delete(idCategoria);
        } catch (IllegalArgumentException e) {
            titulo = "error"; // Captura la excepción de argumento inválido para el mensaje de "no existe"
            detalle = "categoria.error01";
        } catch (IllegalStateException e) {
            titulo = "error"; // Captura la excepción de estado ilegal para el mensaje de "datos asociados"
            detalle = "categoria.error02";
        } catch (Exception e) {
            titulo = "error";  // Captura cualquier otra excepción inesperada
            detalle = "categoria.error03";
        }
        redirectAttributes.addFlashAttribute(titulo, messageSource.getMessage(detalle, null, Locale.getDefault()));
        return "redirect:/categoria/listado";
    }

    @GetMapping("/modificar/{idCategoria}")
    public String modificar(@PathVariable("idCategoria") Integer idCategoria, Model model, RedirectAttributes redirectAttributes) {
        Optional<Categoria> categoriaOpt = categoriaService.getCategoria(idCategoria);
        if (categoriaOpt.isEmpty()) {
            redirectAttributes.addFlashAttribute("error", messageSource.getMessage("categoria.error01", null, Locale.getDefault()));
            return "redirect:/categoria/listado";
        }
        model.addAttribute("categoria", categoriaOpt.get());
        return "/categoria/modifica";
    }
}
```

Creacion del folder js en static

Acá colocaremos un archivo de sentencias javascript que llamaremos rutinas.js este archivo se encuentra en los recursos.



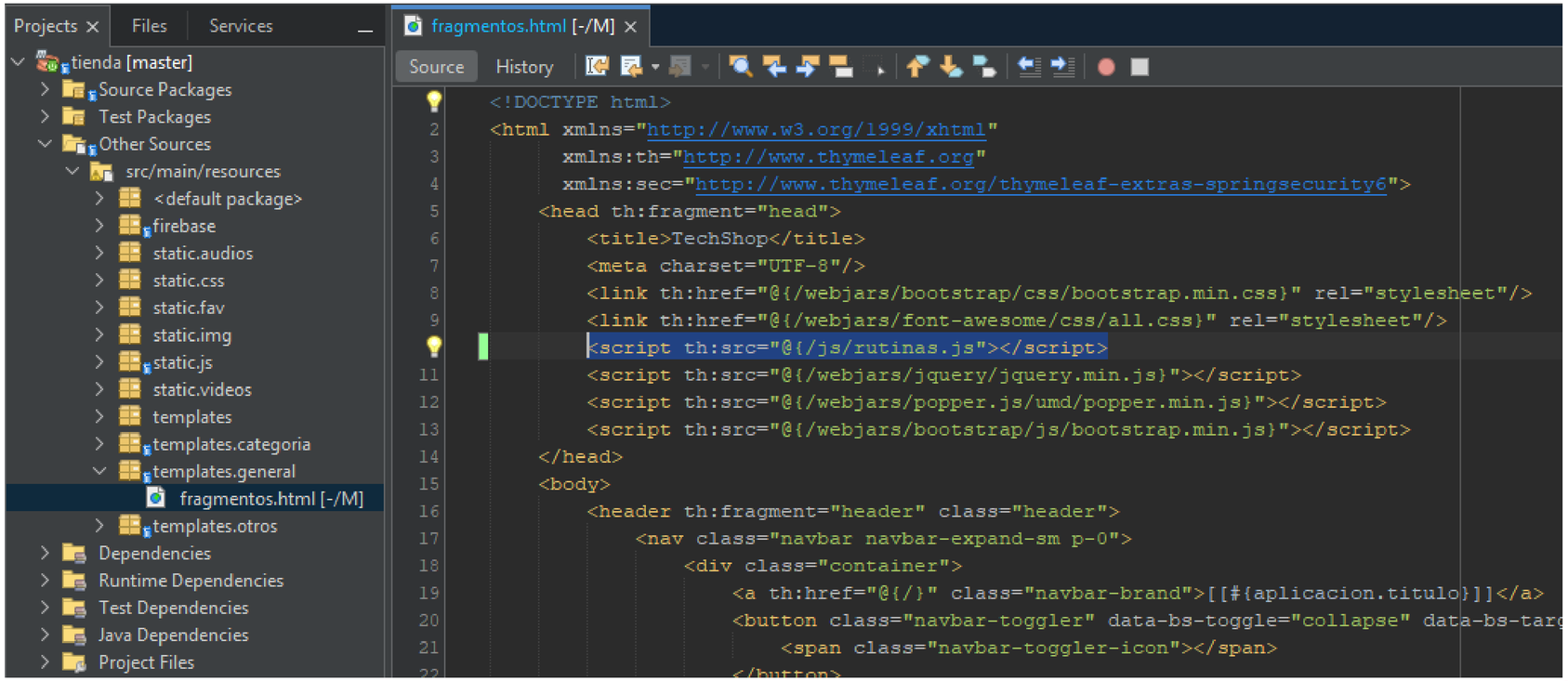




fidélitas Universidad

fragmentos.html

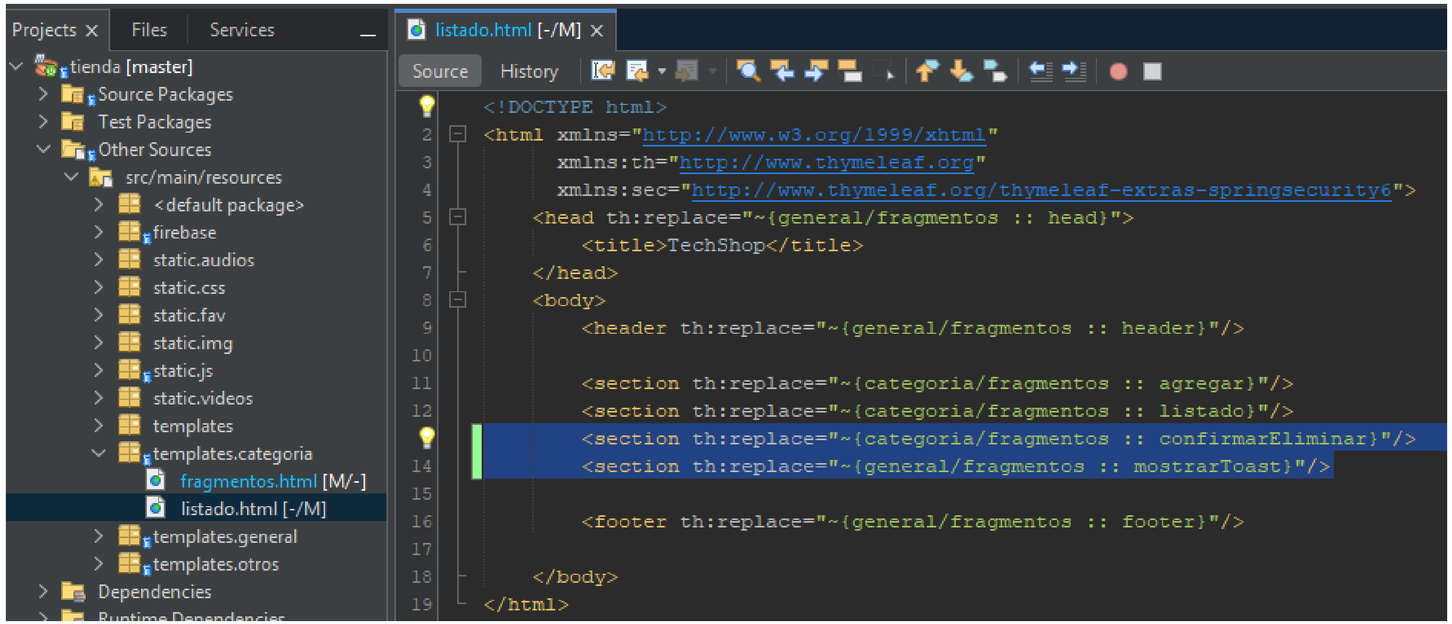
Se incluye un enlace en fragmentos del folder general



fidélitas Universidad

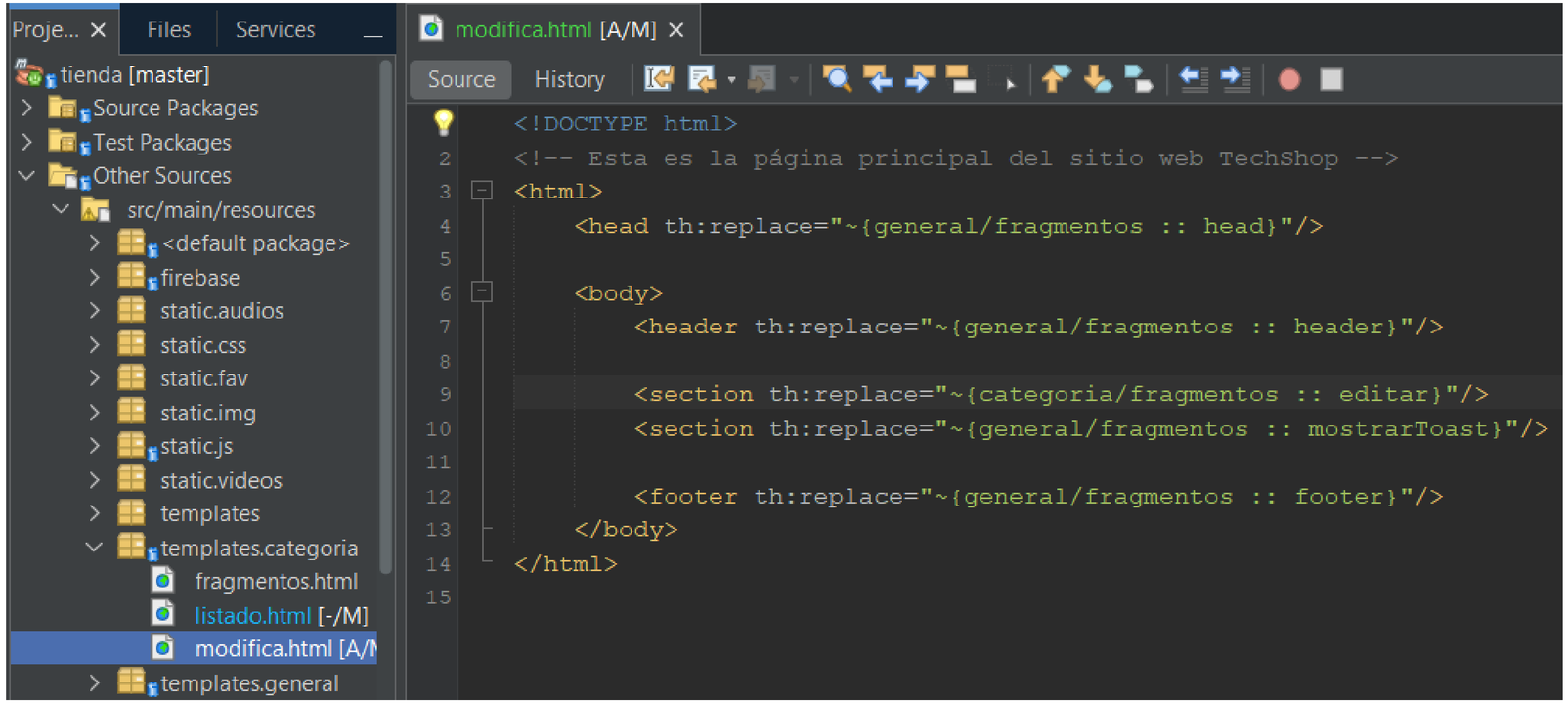
listado.html

Se crean unas líneas en el archivo listado.html



Se crea la página modifica.html

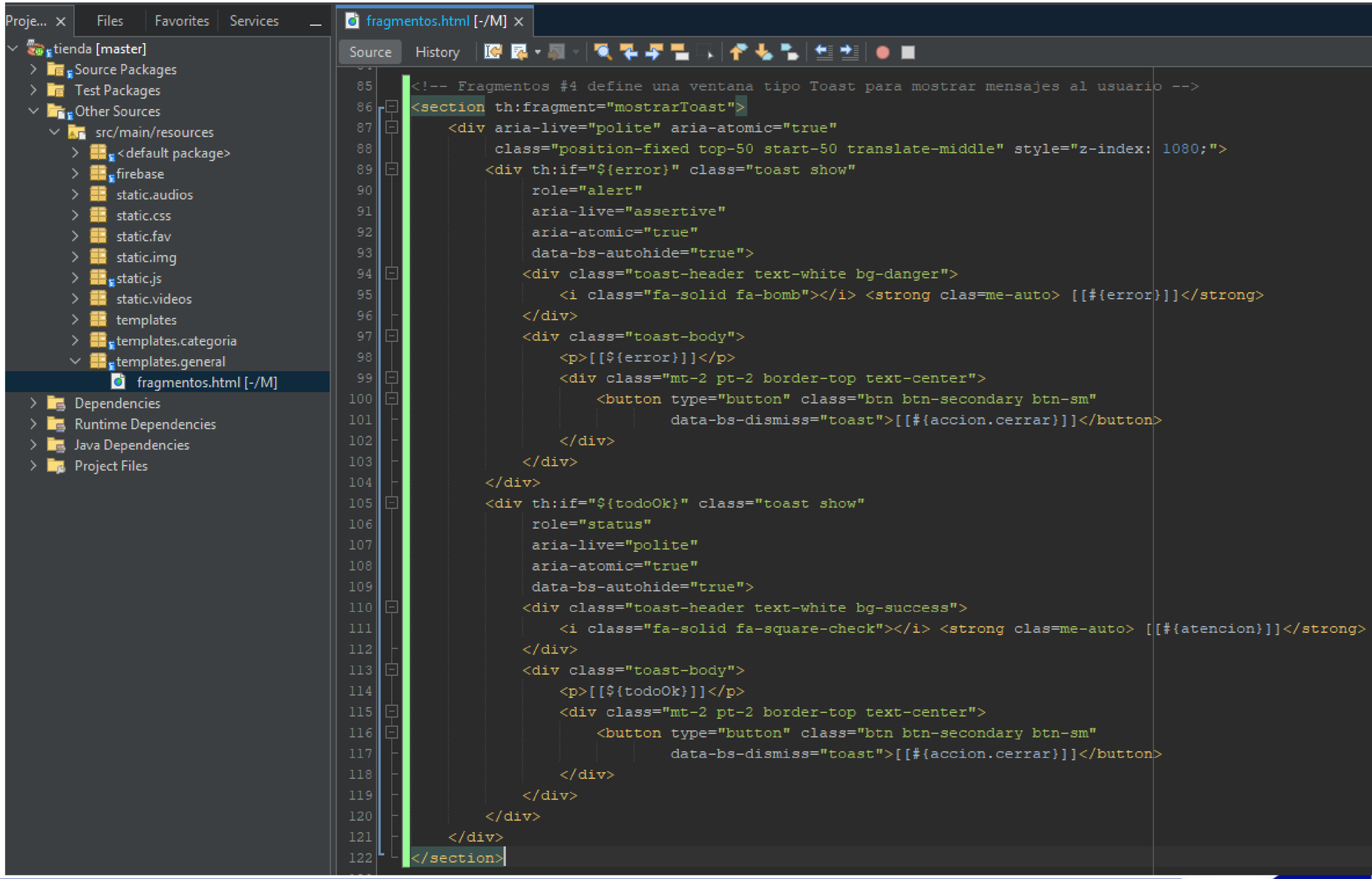
Que mostrará la categoría a modificar, se puede tomar como base el archivo modifica.html



fidélitas Universidad

fragmentos.html

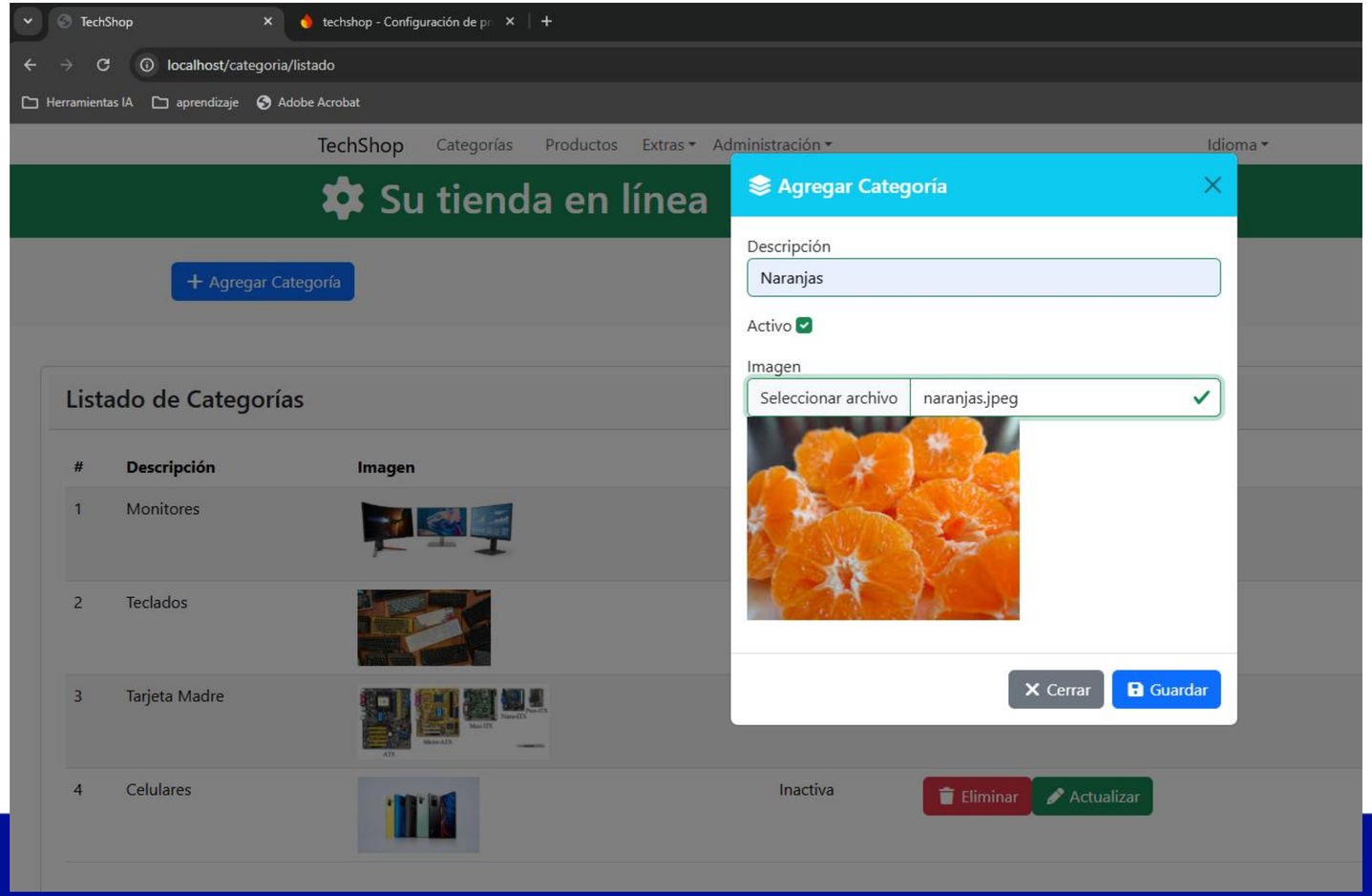
Se crea un fragment en general\fragmentos, se coloca la parte del mostrarToast




```html
<!-- Fragmentos #4 define una ventana tipo Toast para mostrar mensajes al usuario -->
<section th:fragment="mostrarToast">
    <div aria-live="polite" aria-atomic="true"
         class="position-fixed top-50 start-50 translate-middle" style="z-index: 1080;">
        <div th:if="${error}" class="toast show"
             role="alert"
             aria-live="assertive"
             aria-atomic="true"
             data-bs-autohide="true">
            <div class="toast-header text-white bg-danger">
                <i class="fa-solid fa-bomb"></i> <strong clas=me-auto> [[#{error}]]</strong>
            </div>
            <div class="toast-body">
                <p>[[${error}]]</p>
                <div class="mt-2 pt-2 border-top text-center">
                    <button type="button" class="btn btn-secondary btn-sm"
                            data-bs-dismiss="toast">[[#{accion.cerrar}]]</button>
                </div>
            </div>
        </div>
        <div th:if="${todoOk}" class="toast show"
             role="status"
             aria-live="polite"
             aria-atomic="true"
             data-bs-autohide="true">
            <div class="toast-header text-white bg-success">
                <i class="fa-solid fa-square-check"></i> <strong clas=me-auto> [[#{atencion}]]</strong>
            </div>
            <div class="toast-body">
                <p>[[${todoOk}]]</p>
                <div class="mt-2 pt-2 border-top text-center">
                    <button type="button" class="btn btn-secondary btn-sm"
                            data-bs-dismiss="toast">[[#{accion.cerrar}]]</button>
                </div>
            </div>
        </div>
    </div>
</section>
```

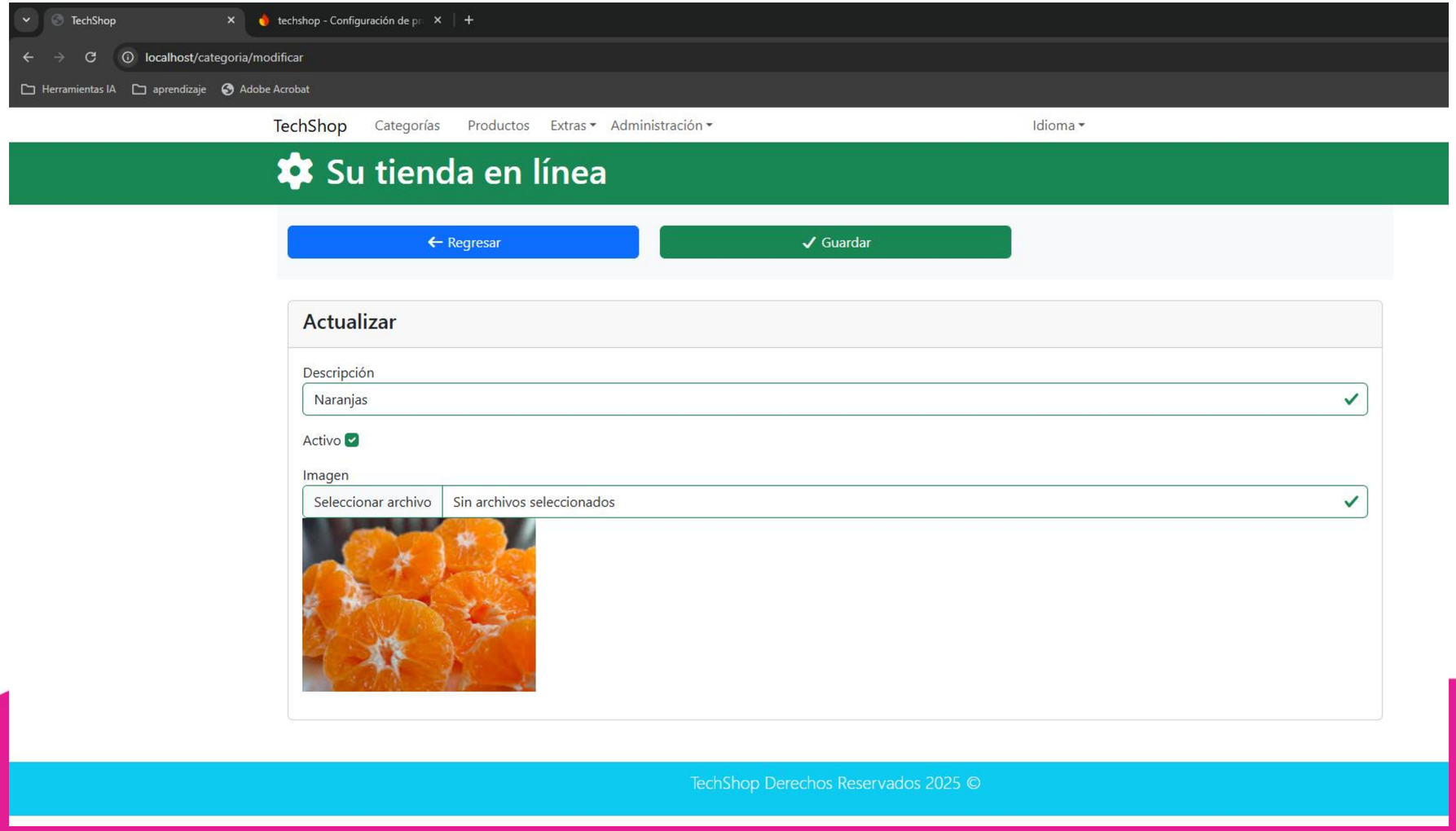
fidélitas Universidad

Agregando una categoria



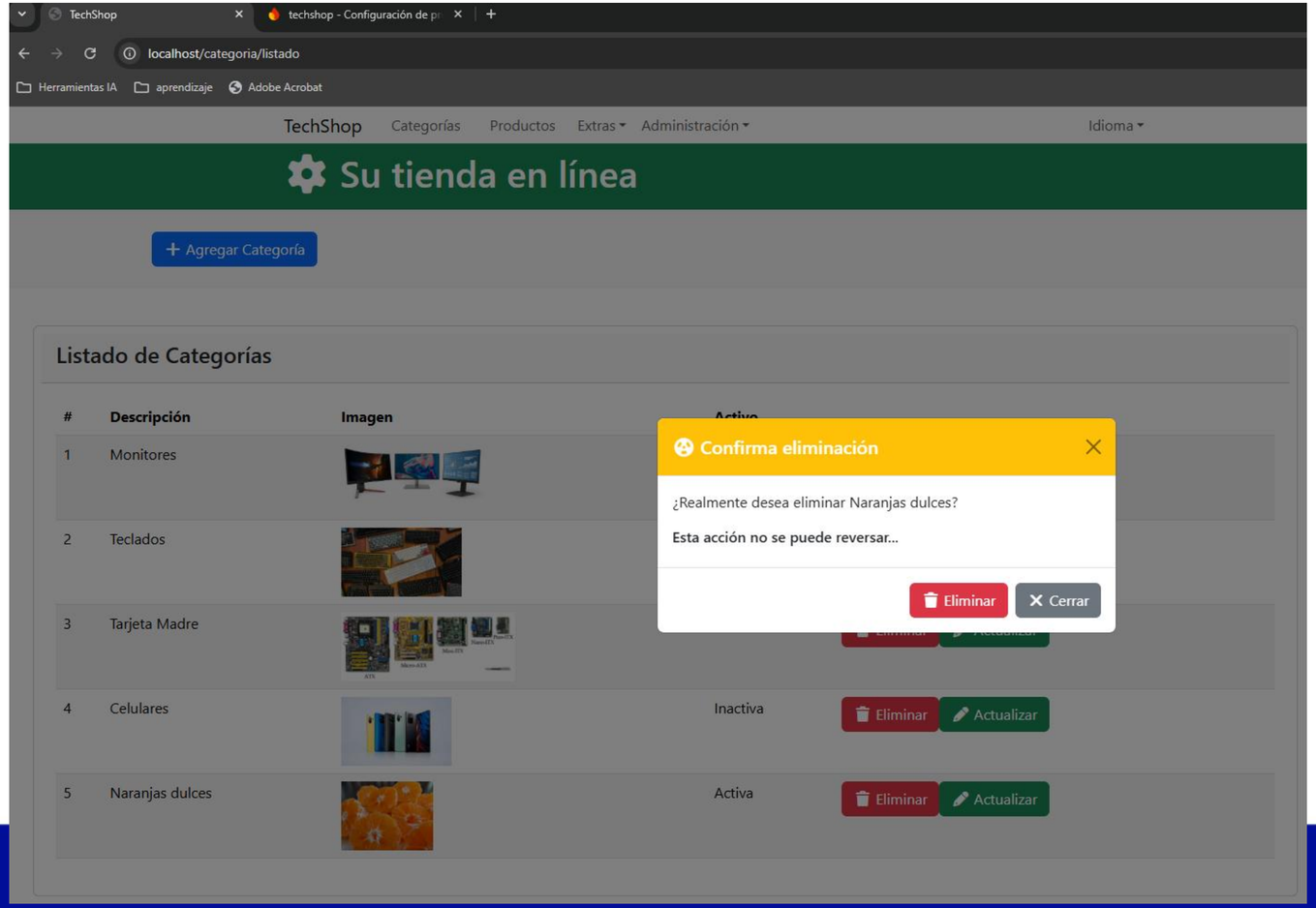
fidélitas
Universidad

Modificando una categoria

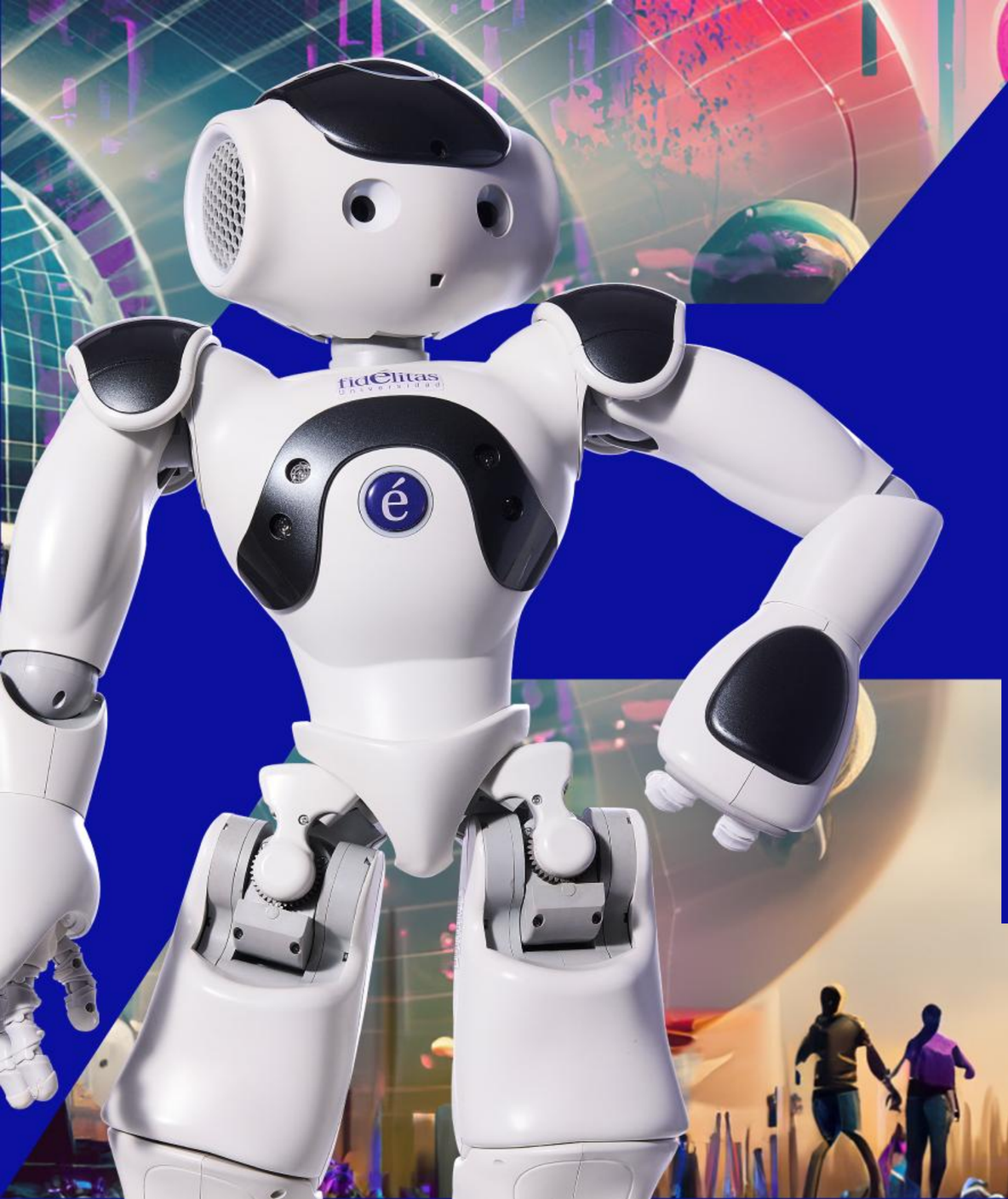


fidélitas Universidad

Eliminando una categoria



fidélitas
Universidad



fidélitas
Universidad

Nos vemos en semana 5

Reto: intenta modificar el esquema de colores que tienen las ventanas, hazlo a tu gusto, cambia el tiempo predeterminado para que se quite la ventana Toast que aparece.

Tip: investiga como usar colores personalizados, revisa cómo cambiar el timer del Toast